

Lab 09 — Whitelisting a Rule ID (so a Real User Isn't Blocked)

Level: Intermediate **Goal:** When CRS blocks a *legitimate* request, whitelist the offending **rule ID** so that user gets through — and understand the trade-off you just made. **Time:** ~30 minutes **Operator focus:** This is the fastest tool in the tuning toolbox. It's also the bluntest, so knowing *when* to reach for it (and when to reach for the scoped exclusion from Lab 07 instead) is the real skill.

1. Theory — "whitelisting" vs "blacklisting"

CRS is a **blocklist**: it describes what *bad* looks like and blocks matches. **Whitelisting** is the opposite instruction — you tell the WAF *"this specific thing is fine, do not block it."*

You whitelist at different granularities:

You whitelist...	Directive	Effect
A rule ID	<code>SecRuleRemoveById 942100</code>	That rule never fires, anywhere
A range of IDs	<code>SecRuleRemoveById 920200-920210</code>	A whole block of rules off
A rule family (tag)	<code>SecRuleRemoveByTag "attack-sqli"</code>	Every rule with that tag off
A trusted client	<code>SecRule REMOTE_ADDR ... ctl:ruleEngine=...</code>	Requests from that source skip enforcement

Whitelisting a rule ID vs. the scoped exclusion of Lab 07: In Lab 07 we used `ctl:ruleRemoveTargetById=942100;ARGS:q` to disable a rule **only for one argument on one path**. `SecRuleRemoveById 942100` disables it **everywhere**. The scoped version is safer; the global version is simpler. Reach for the global one only when the rule is wrong for your app as a *whole* — otherwise prefer the scalpel.

2. Reproduce the block

A legitimate product search contains the words "select ... from", which the SQLi rule `942100` mistakes for an attack. Against the normal stack (`:8080`):

```
curl -s -o /dev/null -w "%{http_code}\n" \  
--get "http://localhost:8080/search" \  

```

```
--data-urlencode "q=Compare and select from our items" # 403
```

Confirm the culprit rule ID from the log (never guess it):

```
docker logs modseclabs-waf | grep -o '"ruleId":"[0-9]*"[^"]*"data":"[^"]*"' | tail -2
# "ruleId":"942100", ... "data":"Matched Data: ... within ARGS:q: Compare and select from
our items"
# "ruleId":"949110", ... msg "Inbound Anomaly Score Exceeded (Total Score: 5)"
```

⚠️ **Two IDs appear — whitelist the right one.** You'll almost always see **949110** in a block ("Inbound Anomaly Score Exceeded"). That is the **anomaly-score gate**, not a detection rule — it fires on every block. **Never whitelist 949110** (it would disable blocking site-wide). The ID you whitelist is the **cause**: the rule whose **data** says *"Matched Data ... within **ARGS:q**"*. If your terminal only shows the **949110** line, scroll/grep for the **ARGS:q** line — the cause is logged separately.

And your cause ID may not be 942100. It depends on your CRS version — on another build the same search may trip **942150** / **942260** /etc. **Take the ID from your "Matched Data" line, not from this sheet.** (If you whitelist **942100** and the block persists, you excluded the wrong ID — that is exactly this mistake.)

Here the block came from a **single** scoring rule (**942100** , +5) and the threshold is 5, so whitelisting just that one rule drops the score to **0** and the anomaly gate stops triggering. The rule ID to whitelist here is **942100** — substitute whatever your log shows.

3. Whitelist the rule ID

Create **rules/WHITELIST.conf** :

```
# Whitelist a single rule ID globally.
SecRuleRemoveById 942100

# You can also whitelist a RANGE (inclusive):
SecRuleRemoveById 920200-920210

# ...or every rule carrying a tag (powerful -- usually too broad):
# SecRuleRemoveByTag "attack-sqli"
```

Load order matters (again). **SecRuleRemoveById** must be processed **after** the rule it targets has been *defined* but its removal applies at config load. The safe, conventional home is a **REQUEST-900-*** file, which loads before the **9xx** detection rules — CRS handles the removal correctly from there.

Start a WAF with the whitelist mounted:

```
docker run -d --name modseclabs-waf-wl \
--add-host=host.docker.internal:host-gateway \
```

```
-e BACKEND="http://host.docker.internal:5000" -e PORT=8080 \  
-e MODSEC_RULE_ENGINE=On -e PARANOIA=1 -e ANOMALY_INBOUND=5 \  
-e MODSEC_AUDIT_LOG=/dev/stdout -e MODSEC_AUDIT_ENGINE=RelevantOnly \  
-v "$PWD/rules/WHITELIST.conf:/etc/modsecurity.d/owasp-crs/rules/REQUEST-900-  
WHITELIST.conf:ro" \  
-p 8095:8080 owasp/modsecurity-crs:nginx  
sleep 6
```

Always check it loaded cleanly:

```
docker logs modseclabs-waf-wl | grep -iE "error|emerg" # (no output = good)
```

4. Verify — the user is unblocked, security mostly intact

```
curl -s -o /dev/null -w "legit search (was 403) -> %{http_code}\n" \  
--get "http://localhost:8095/search" --data-urlencode "q=Compare and select from our  
items"  
curl -s -o /dev/null -w "real UNION SQLi on q -> %{http_code}\n" \  
--get "http://localhost:8095/search" --data-urlencode "q=1' UNION SELECT password FROM  
users--"  
curl -s -o /dev/null -w "XSS still blocked -> %{http_code}\n" \  
--get "http://localhost:8095/search" --data-urlencode "q=<script>alert(1)</script>"  
curl -s -o /dev/null -w "clean request -> %{http_code}\n" \  
"http://localhost:8095/"
```

Result:

```
legit search (was 403) -> 200  
real UNION SQLi on q -> 403  
XSS still blocked -> 403  
clean request -> 200
```

After whitelisting rule 942100: legit search returns 200 (was 403) [HTTP 200]

Results for: Compare and select from our items

No results found.

Prove the **anomaly gate no longer triggers** for that request — after the whitelist there should be **no** "Score Exceeded" event for the benign search:

```
curl -s -o /dev/null --get "http://localhost:8095/search" \
  --data-urlencode "q=Compare and select from our items"
docker logs --since 10s modseclabs-waf-wl | grep -c "Score Exceeded" # -> 0
```

The count is **0**: with the sole contributor (**942100**) removed, the score for that request is 0, so **949110** never fires. That is what "the whitelist stops the anomaly trigger" looks like — the request isn't just tolerated, it no longer scores at all.

Read that result carefully — this is the whole lesson. Whitelisting **942100** let the honest search through, **and** a real **UNION SELECT** on the *same* parameter is *still* blocked. Why? Because that attack also trips **other** SQLi rules (**942190** , **942360** , ...) and the anomaly score still crosses the threshold. Removing one rule from a scoring system rarely opens a real hole — but it *does* remove one layer, so you accept a small, known risk.

5. Whitelisting a trusted client (bonus)

Sometimes you don't want to whitelist a *rule* — you want to trust a *source* (an internal QA scanner, an uptime probe, a partner's server) so its requests are never blocked. Create `rules/ALLOWLIST-IP.conf` :

```
SecRule REMOTE_ADDR "@ipMatch 172.17.0.1" \
  "id:1900010,phase:1,pass,nolog,\
  msg:'ModSecLabs: trusted source, engine set to DetectionOnly',\
  ctl:ruleEngine=DetectionOnly"
```

Mount it and, from that trusted IP, even an attack passes — but is **still logged** (we used `DetectionOnly` , not `Off` , so you keep visibility):

```
curl -s -o /dev/null -w "XSS from trusted IP -> %{http_code}\n" \
  --get "http://localhost:8096/search" --data-urlencode "q=<script>alert(1)</script>"
# 200 (allowed), and 941100 still appears in docker logs
```

Danger — allowlist the narrowest source you can. `REMOTE_ADDR` is easy to **spoof** if your WAF sits behind a proxy/load-balancer that sets `X-Forwarded-For` , because then every request appears to come from the proxy's IP. Only trust `REMOTE_ADDR` when the WAF is the true edge, and prefer `ctl:ruleEngine=DetectionOnly` over `Off` so you never go fully blind.

Clean up

```
docker rm -f modseclabs-waf-wl modseclabs-waf-ip 2>/dev/null
```

6. When to whitelist a rule ID vs. other tools

Situation	Best tool
One rule is wrong for your whole app	<code>SecRuleRemoveById <id></code> (this lab)
One rule misfires on one field / one path	<code>ctl:ruleRemoveTargetById</code> (Lab 07)
A whole rule family is irrelevant to your stack	<code>SecRuleRemoveByTag</code>
A specific trusted source must never be blocked	IP allowlist + <code>ctl:ruleEngine</code>
The rule is right but too twitchy globally	raise the anomaly threshold (Lab 05)

Golden rule of whitelisting: whitelist the **narrowest** thing that fixes the problem, always **from the audit log**, and always **re-test that real attacks are still blocked** afterwards. A whitelist is a permanent hole you punched on purpose — make it as small as the problem, and no bigger.

7. Recap

- Whitelisting tells the WAF *not* to block something it otherwise would.
- `SecRuleRemoveById 942100` disables a rule **globally** — simple but blunt.
- Thanks to anomaly **scoring**, removing one rule usually leaves real attacks blocked by the others — but it's still a deliberate risk you own.
- You can also whitelist ranges, tags, or a **trusted client IP** (prefer `DetectionOnly` so you keep logging).
- Pick the **narrowest** mechanism, drive it from the **log**, and **re-test**.